

```
eth->h_source[3] = (unsigned char)(ifreq_c.ifr_hwaddr.sa_data[3]);
eth->h_source[4] = (unsigned char)(ifreq_c.ifr_hwaddr.sa_data[4]);
eth->h_source[5] = (unsigned char)(ifreq_c.ifr_hwaddr.sa_data[5]);

/* filling destination mac. DESTMAC0 to DESTMAC5 are macro
having octets of mac address. */
eth->h_dest[0] = DESTMAC0;
eth->h_dest[1] = DESTMAC1;
eth->h_dest[2] = DESTMAC2;
eth->h_dest[3] = DESTMAC3;
eth->h_dest[4] = DESTMAC4;
eth->h_dest[5] = DESTMAC5;

eth->h_proto = htons(ETH_P_IP); //means next header will be
IP header

/* end of ethernet header */
total_len+=sizeof(struct ethhdr);
```

Constructing the IP header

To construct the IP header, increment *sendbuff* by the size of the Ethernet header and fill each field of the *iphdr* structure. Data after the IP header is called the payload for the IP header and, in the same way, data after the Ethernet header is called the payload for the Ethernet header. In the IP header, there is a field called *Total Length*, which contains the size of the IP header plus the payload. To know the size of the payload of the IP header, you must know the size of the UDP header and the UDP payload. So, some field of the *iphdr* structure will get the value after filling the UDP header field.

```
struct iphdr *iph = (struct iphdr*)(sendbuff + sizeof(struct
ethhdr));
iph->ihl = 5;
iph->version = 4;
iph->tos = 16;
iph->id = htons(10201);
iph->ttl = 64;
iph->protocol = 17;
iph->saddr = inet_addr(inet_ntoa(((struct sockaddr_in
*)&(ifreq_ip.ifr_addr))->sin_addr));
iph->daddr = inet_addr("destination_ip"); // put destination
IP address
```

```
total_len += sizeof(struct iphdr);
```

Construct the UDP header

Constructing the UDP header is very similar to constructing the IP header. Assign values to the fields of the *udphdr* structure. For this, increment the *sendbuff* pointer by the size of the Ethernet and the IP headers.

```
struct udphdr *uh = (struct udphdr *)(sendbuff +
sizeof(struct iphdr) + sizeof(struct ethhdr));
```

```
uh->source = htons(23451);
uh->dest = htons(23452);
uh->check = 0;
```

```
total_len+= sizeof(struct udphdr);
```

Like the IP header, the UDP also has the field *len*, which contains the size of the UDP header and its payload. So, first, you have to know the UDP payload, which is the actual data that will be sent.

Adding data or the UDP payload

We can send any data:

```
sendbuff[total_len++] = 0xAA;
sendbuff[total_len++] = 0xBB;
sendbuff[total_len++] = 0xCC;
sendbuff[total_len++] = 0xDD;
sendbuff[total_len++] = 0xEE;
```

Filling the remaining fields of the IP and UDP headers

We now have the *total_len* pointer and with the help of this, we can fill the remaining fields of the IP and UDP headers:

```
uh->len = htons((total_len - sizeof(struct iphdr) -
sizeof(struct ethhdr)));
```

```
struct ifreq {
    char ifr_name[IFNAMSIZ]; /* Interface name */
    union {
        struct sockaddr ifr_addr;
        struct sockaddr ifr_dstaddr;
        struct sockaddr ifr_broadaddr;
        struct sockaddr ifr_netmask;
        struct sockaddr ifr_hwaddr;
        short ifr_flags;
        int ifr_ifindex;
        int ifr_metric;
        int ifr_mtu;
        struct ifmap ifr_map;
        char ifr_slave[IFNAMSIZ];
        char ifr_newname[IFNAMSIZ];
        char ifr_data;
    };
};
```

Figure 11: Structure of *ifreq*

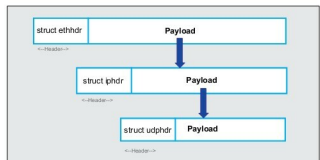


Figure 12: Graphical representation of packets with their structure and payload